Next-Gen Read Alignment and Contig Query Identification

By: Christian Anderson

05 May 2026

Contents

Introduction

Sequencing a genome is not straightforward. Due to the limitations of our technology and a genome's sheer size, the best sequencing technologies still need fragments. Genomes can be billions of base pairs long, but sequencing machines can only read hundreds to thousands at a time, so the genome must first be shattered into millions of small fragments and then sequenced independently. These short sequences, or reads, are then computationally reassembled. The first technologies, including Sanger sequencing, were tedious and time consuming; while accurate, Sanger could only produce ~1,000 bp reads and required manual processing. Next-generation platforms like Illumina parallelized the process massively, reducing cost and time by orders of magnitude. Short reads are highly accurate but struggle with repetitive regions, introducing a new trade-off. Long-read platforms like PacBio and Oxford Nanopore are able to span these regions but tend to trade accuracy for length, favoring completeness over correctness.

Sequencing does not reconstruct the original sequence. The outputs of these technologies are fragments, including many duplicates, which must be reconstructed into the original sequence. This can be done using a reference genome which acts as a template, or it can be done *de novo*, using only the fragments' overlap with each other to map them back together. A recent example of this was during the COVID-19 pandemic when there was no reference genome to use in vaccine development. Researchers sequenced the ~30 kb Sars-Cov-2 genome, enabling vaccine design. There are also many applications where a researcher may have the sequence from a region of interest and only cares about the region surrounding it, for example when validating a knock-out model or identifying contamination in the samples' sequences.

In this scenario an algorithm would need to identify the largest contiguous containing a query of interest from the set of next-generation reads sequenced using the aforementioned technologies. This is the scenario this report explores.

Overview of Current Methodologies

Read alignment boils down to two approaches, overlap layout consensus (OLC) and De Brujin graphs (DBG). OLC looks at the raw sequencing reads and finds how each one overlaps with every other one. This can result in accurate contiguous sequences (contigs) as the reads are not processed prior to alignment, but the number of comparisons between each read makes this method computationally inefficient, solvable in $O(N^2)$ time. The DBG approach is more efficient ($O(N)$) but less accurate and are more prone to repetitive regions and duplication. This approach uses kmer fragments from each read to compare overlap where the first $k-1$ letters of one kmer is compared to the last $k-1$ of another. Similar to OLC, DBG construction compares string overlap between each kmer, but because each fragment is the same length, one-to-one comparison is possible. The result is a graph where each node is a kmer and edges indicate directionaly overlap between nodes. In both scenarios, the original sequence (or fragment containing the query) is constructed by traversing the graph, connecting each read/kmer along the way. Unguided, the only way to do this would be to brute-force traverse every possible path, but since we have a query, we only care about paths spanning out from our query of interest. This means that a comparatively few connections need to be explored. The next step of *exploring* out from the query

sequence (which is essentially a cloud of kmers which originated from the query) can proceed in one of two ways. The first is depth-first search (DFS) which explores one path at a time until it terminates, recording but ignoring branching paths as it explores. The other is breadth-first search (BFS) which does the opposit. This method explores every path one step at a time, building many contiguous sequences in parallel. The DFS is often preferred over BFS as it does not require every path to be stored in memory.

My Approach

In my approach, I use depth-first search, De Brujin graph exploration where I iteratively explore paths one at a time, storing the current *state* in a list containing the current node, the current growing contig, and the nodes visited up to that point (unique to that contig). While my algorithm passes through these states (explores the paths), it also records the predecessor nodes, successor nodes, and kmer-to-read mapping (read id and region with in the read). In early iterations I filled a directed adjacency matrix where a non-zero cell at position [i, j] represented a directed edge from kmer_i to kmer_j, meaning the last k−1 characters of kmer_i match the first k−1 characters of kmer_j. For example, “fooba” → “oobar” would be recorded as a non-zero entry at [i, j] where i is the index of “fooba” and j is the index of “oobar”. Building the adjacency matrix naturally used considerable memory and required more time to manipulate and query. I also tried using a recursive approach where the ‘explore next’ functionality was a recursive function which recursively passed the current node, contig, and visited nodes. It was elegant, but very difficult to debug, so I settled on the listed state approach. My algorithm also builds on the previous iteration by short-cutting graph traversal by starting at the query sequence. I think this was the improvement had the most impact on this algorithm’s overall performance.

The algorithm first identifies the region to explore out from. This consists of the kmers created by the query. Next, the algorithm separately explores both left and right from the contig where each time the contig grows larger in its respective direction.

inaholeinthegroundtherelivedahobbit

Taking the above sentence as an example genome with the query ‘ground’, the algorithm would first find the kmers that compose the query. Using a k of 3 these would be gro, rou, oun, and und.

Starting with the rightmost kmer of the core cloud, “und”, the algorithm then searches for kmers whose first $k-1$ characters match the last $k-1$ characters of “und”, i.e. any kmer starting with “nd”. In this genome that yields “ndt”, which becomes the next node. The algorithm continues: “ndt” → “dth” → “the” → “her” → “ere”, extending the right contig one character at a time until no successor exists. Simultaneously, starting from the leftmost query kmer “gro”, the algorithm searches in the opposite direction — looking for kmers whose last $k-1$ characters match the first $k-1$ characters of “gro”, i.e. any kmer ending in “gr”. That yields “egr”, then “heg” → “the” → “nth” → “int” → “ein” →..., extending the left contig backward toward the beginning of the genome. The two contigs are then joined around the query to reconstruct the full sequence: ...einthe**ground**there...

The meat of this process executed using the `make_contigs()` function in the `functions.py` script here.

There are currently 3 hyper-parameters used in my pipeline. The first and most important is `k`, which specifies the size of each kmer. Small values of `k` lead to fewer possible sequence combinations (4^k), so the same kmer appears across many unrelated reads, creating spurious edges in the graph. As `k` increases, combinations increase exponentially and unique kmers peak where nearly all possible sequences are observed. Beyond this peak, unique kmers decline gradually as longer kmers become increasingly rare across reads, with fewer surpassing the frequency threshold. This decline decelerates as `k` approaches read length, at which point kmers can no longer be fully contained within a single read. At this point the unique kmer count falls to zero and the graph fragments. The optimal `k` therefore balances uniqueness against frequency. I chose $k = 19$ to increase the number of possible connections that could lead to real contigs. See *Figure 1* below.

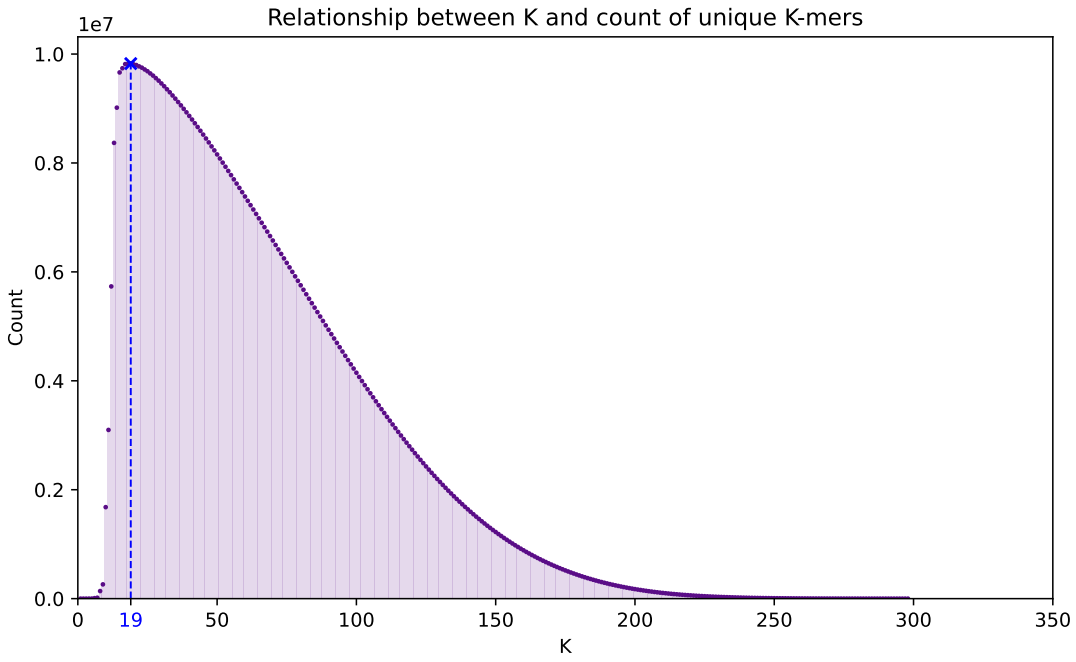

Figure 1: Relationship between k and unique K-mer count

The second parameter is `t` which excludes the kmers seen infrequently enough to likely be due to sequencing error. It is also important to strike a balance with t; if `t` is too high, only the predominant signal will be retained, if it is too low, much of the signal will be due to noise.

The last parameter, `max_visits` controls how many times the 'explorer' is allowed to visit a particular kmer along a particular path. This is crucial to my algorithm not getting stuck in cycles, but if `max_visits` is too high, the 'explorer' may loop through the same node more times than is necessary. For this reason `max_visits` has been fixed at a value of 2.

Results

Tests

To make the development of this method more manageable I constructed three test scenarios which get progressively more complex. The first test constructs the string

STRTABCDEFGABCHIJKDEFENDSTRTABCDEFGABCHIJKDEFEND

using **STRT** as the query. This test explores how my algorithm behaves in the edge case (literally) where the query is at the beginning of the sequence. I simulate reads from the sequence by randomly cutting k-length mers with duplication.

The second test uses the first sentence of the hobbit as a more complex sequence and a central segment as the query. The test was framed to explore cycles in more depth, specifically at 'hobbit' related nodes, as this word repeats frequently. To explore all possible paths radiating out from the query, it was necessary to visit nodes more than once, hence the introduction of the `max_visits` parameter.

Finally, I simulated a sequence and randomly segmented it into fake reads. Using this I was able able to explore performance at scale using the four-letter alphabet (ATCG).

I additionally included unit tests which test each function against correct output for toy examples; I also integrated these in the GitHub page for this repo.

Real Sequence

Finally, I attempted to assemble the assigned genome using the pipeline I developed. I ran my algorithm on the provided fasta file with the query sequence and identified a the following longest contig:

TTCTCCTCTTCCTCATCTCCGGCCTTTCGACCTGCAGCCAATAT**GGGATCGGCCATTGAACAAGATGGATTGCACGCAGG**
TTCTCCGGCCGCTTGGGTGGAGAGGCTATTCGGCTATGACTGGGCACAACAGACAATCGGCTGCTCTGATGCCGCCGTGT
TCCGGCTGTCAGCGCAGGGGCGCCCGGTTCTTTTTGTCAAGACCGACCTGTCCGGTGCCCTGAATGAACTGCAGGACGAG
GCAGCGCGGCTATCGTGGCTGGCCACGACGGGCGTTCCTTGCGCAGCTGTGCTCGACGTTGTCACTGAAGCGGGAAGGGA
CTGGCTGCTATTGGGCGAAGTGCCGGGGCAGGATCTCCTGTCATCTCACCTTGCTCCTGCCGAGAAAGTATCCATCATGG
CTGATGCAATGCGGCGGCTGCATACGCTTGATCCGGCTACCTGCCCATTCGACCACCAAGCGAAACATCGCATCGAGCGA
GCACGTACTCGGATGGAAGCCGGTCTTGTCGATCAGGATGATCTGGACGAAGAGCATCAGGGGCTCGCGCCAGCCGAACT
GTTCGCCAGGCTCAAGGCGCGCATGCCCGACGGCGATGATCTCGTCGTGACCCATGGCGATGCCTGCTTGCCGAATATCA
TGGTGGAAAATGGCCGCTTTTCTGGATTCATCGACTGTGGCCGGCTGGGTGTGGCGGACCGCTATCAGGACATAGCGTTG
GCTACCCGTGATATTGCTGAAGAGCTTGGCGGCGAATGGGCTGACCGCTTCCTCGTGCTTTACGGTATCGCCGCTCCCGA
TTCGCAGCGCATCGCCTTCTATCGCCTTCTTGACGAGTTCTTCTGAGGGGATCCGCTGGGAGTTCAAAGGAGGGGTCCCC
TATCGAATTACAGTGACTGCAGTATATTCTGGAGGATTAGCTGCTGCACCCTCAGTTTGGGA

I queried this sequence to the BLASTn database with no and it aligns with the *Mus musculus* transgenic G610C Neo+ founder type I procollagen alpha2 chain (Col1a2) gene, Col1a2-G610C allele, exon 33 and partial cds sequence. I also queried for just human sequences and found significant a significant hit against the humanA SGK1 S422D gene for serum/glucocorticoid regulated kinase 1. Most likely this sequence comes from Mus musculus, given the stronger and more specific alignment to the mouse Col1a2-G610C transgenic allele. The human SGK1 hit likely reflects conserved regulatory coding sequence shared between species rather than full human origin. The Col1a2 gene encodes a structural component of type I collagen, and the mutation is a common disease model of osteogenesis imperfecta.

Discussion

De novo assembly is difficult. The most fundamental challenge is that there is no ground truth against which to validate results, as there would be with a reference. My approach addresses this by anchoring assembly to the query sequence, which constrains the search space and avoids the need for an exhaustive graph search. However, this also means the algorithm is only as good as the reads supporting the region around the query because sparse or erroneous coverage would truncate or corrupt the contig. The three hyper-parameters (`k`, `t`, `max_visits`) each introduce a bias-variance tradeoff. Without a ground truth, tuning these parameters relied testing performance rather than external validation. Applying my pipeline to the assigned sequence I found a contig that BLASTn aligned confidently to the Mus musculus Col1a2-G610C transgenic allele which suggests the reads were derived from a mouse osteogenesis imperfecta model. The assembly recovered the full query and extended meaningfully in both directions, which is encouraging given the algorithm's simplicity. To improve my method further, I would incorporate edge weighting based on kmer frequency. This addition would cause the traversal to preferentially traverse through the more common nodes, thereby hopefully avoiding the infrequent, error-prone nodes. Additionally, I would try to find a way to escape cycles dynamically without needing the hard `max_visits` threshold.